

Turbo Charge Python Apps with Speed



Python's ease of use, friendliness, 'batteries-included' huge standard library, plus add-on libraries, make it one of the most popular languages in the FOSS world. However, as a byte-code-compiled language, it loses out on the performance front in comparison with C and C++. This article presents several possible solutions to this problem, so you can still use Python's familiar syntax, yet achieve better, or even native-code performance for programs that need it.

Python is undoubtedly one of the most popular programming languages ever created. It has a marvellous syntax, and great libraries supporting almost all the areas of computing that one can think of. Python provides great programming productivity and flexibility—but it loses out on the performance front. It is a byte-code-compiled dynamic programming language that executes through its virtual machine, so the performance of Python programs is much slower compared to native-code-compiled programming languages like C, C++, etc. Such languages are wonderful for program performance, but greatly hamper programming productivity and flexibility.

Another problem with Python is that its current standard implementation is quite unable to manipulate the multiple processing cores of the now-standard multi-core processors.

These speed-dampening factors in Python are often frustrating to developers, since they have to switch to writing complicated C or C++ compiled code chunks to mitigate the performance drawbacks of Python.

Thanks to the FLOSS community, however, there are many ways to turbo-charge your Python programs without resorting to writing complicated and time-consuming C or C++ compiled code chunks. These solutions are in the form of Python JIT compilation modules like Psyco, alternative implementations of standard